

Retrouvez l'ordre

- **Limite de temps** : 10 minutes
- **Environnement** : un GPU (≈ 16 GB VRAM), sans accès à internet
- **Taille de la solution** : `solution.ipynb` ≤ 1 MB
- **Stockage** : 5 GB

Problème

Vous disposez de dialogues en anglais parlé entre deux participants (ou locuteur), *Speaker A* et *Speaker B*. Chaque dialogue est segmenté en tours de parole, chaque tour contenant la parole d'un seul locuteur. Chaque tour est stocké dans un fichier audio `.wav` distinct ; un dialogue complet est donc représenté par un ensemble de fichiers `.wav`, un pour chaque tour.

Malheureusement, les tours ont été mélangés aléatoirement, de sorte que la conversation n'a plus de sens. Dans le nom de fichier `chunk_{k}.wav`, k désigne le k -ième segment (ou *chunk*) de l'ensemble mélangé, et non le k -ième tour du dialogue d'origine.

!! Votre tâche consiste à reconstituer l'ordre chronologique d'origine de la conversation.



Dataset

Chaque dialogue contient n fichiers audio nommés `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. Les segments (ou *chunk*) sont des tours de parole individuels. Les noms de fichiers correspondent uniquement à l'ordre mélangé. Ils n'indiquent pas la place d'un segment (ou *chunk*) dans la conversation d'origine. Chaque dialogue comporte 7–20 segments (ou *chunk*), en mono, à 44.1 kHz (vous pouvez ré-échantillonner).

`prefix.json` contient les indices des noms de fichiers des deux premiers segments (ou *chunk*) de chaque dialogue. Cela identifie le véritable début du dialogue et élimine l'ambiguïté entre la lecture de la conversation dans le sens chronologique ou dans le sens inverse.

Par exemple : 11: [7, 12] signifie que les premier et deuxième tours du dialogue 11 sont respectivement `chunk_7.wav` et `chunk_12.wav`.

Ce qui vous est fourni

Vous recevez **deux dossiers de format identique** :

Dossier	Dialogues	<code>answers.json</code> ?	Utiliser le pour
<code>dataset/train/</code>	1,288	✓ inclus	entraîner / affiner (<i>fine-tune</i>) votre modèle
<code>dataset/test_public/</code>	100	✓ inclus	exécuter votre pipeline et calculer vous-même votre score en local

Lors de l'évaluation, votre dossier `dataset/test_public/` est remplacé de manière transparente par un `hidden evaluation set` (`test_leaderboard_a` pour le classement public et `test_leaderboard_b` pour le classement final) — ceux-ci ont la même taille et le même format que `dataset/test_public/`, mais sans le fichier `answers.json`.

Votre notebook est exécuté à nouveau sur ces données, et le fichier `answers.json` qu'il produit est utilisé pour le calcul du score. Les dialogues de test mis de côté proviennent de la même distribution que `train` ; votre score `test_public` local constitue donc un aperçu fidèle.

Structure des répertoires

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

Sortie

Pour chaque dialogue, déterminez l'ordre chronologique d'origine de ses segments audio (ou *chunk*). Votre prédiction doit être une permutation P de $\{0, 1, \dots, n-1\}$, où $P[i]$ est la position chronologique prédite de `chunk_i.wav` (0 = premier).

Votre fichier de sortie `answers.json` doit associer chaque identifiant de dialogue (ID) à la permutation prédite correspondante :

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

Exemple

Un dialogue comporte 3 segments (ou *chunk*) mélangés `chunk_0`, `chunk_1`, `chunk_2` :

segment (ou <i>chunk</i>) mélangé	contenu parlé	position réelle (rang)
chunk_0.wav	"No worries — I'll send you the notes afterwards."	2 (dernier)
chunk_1.wav	"Hey, are you coming to the three o'clock meeting?"	0 (premier)
chunk_2.wav	"I can't — I've got a dentist appointment then."	1

L'ordre réel est **chunk_1** → **chunk_2** → **chunk_0** ; ainsi, $P = [2, 0, 1]$, et `prefix.json` contient `[1, 2]`.

⚠ P doit être une véritable permutation : de longueur n , indexée à partir de 0, chaque valeur apparaissant exactement une fois. Les doublons, les valeurs manquantes ou hors limites (par exemple, une indexation à partir de 1) donnent un score de 0 pour ce dialogue, tout comme l'absence d'un dialogue dans le fichier. **Un fichier mal formaté ou qui n'est pas au format JSON est rejeté.**

Évaluation

La métrique d'évaluation de cette tâche est **l'exactitude de l'ordre par paires**. Elle examine chaque paire de segments (ou *chunk*) et pose la question : *lequel des deux doit venir en premier ?* Une paire est correcte si votre prédiction donne la même réponse que la vérité terrain. Pour un dialogue comportant n segments (ou *chunk*), il existe

$$M = n(n - 1)/2$$

paires ; soit I le nombre d'inversions — les paires ordonnées différemment de la vérité terrain :

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

i Le score final est la moyenne des scores par dialogue sur l'ensemble des dialogues du split (la base de test par exemple).

Modèles autorisés

Vous pouvez uniquement utiliser les modèles pré-entraînés suivants pour résoudre cette tâche, aussi bien pendant l'entraînement que durant l'évaluation (prédiction ou inférence). Tous ces modèles sont déjà téléchargés et disponibles dans l'environnement. Vous trouverez des exemples d'utilisation dans le notebook `baseline solution.ipynb`. Veuillez noter que vous ne pouvez utiliser aucun autre modèle et que vous n'avez pas accès à internet.

- **Représentations de la parole : wav2vec 2.0.** L'encodeur **Whisper** peut également être utilisé comme extracteur de caractéristiques. [Fiche du modèle wav2vec](#)
- **Reconnaissance automatique de la parole (ASR) : OpenAI Whisper** (toute taille). [Fiche du modèle Whisper](#)
- **Modèle de langage : Qwen2.5-0.5B**, qui peut être utilisé soit en zero-shot, soit affiné sur le split `train` fourni. [Fiche du modèle Qwen](#) Notez que la limite de 10 minutes doit couvrir toute la phase d'entraînement ou de *fine-tuning* (affinement) que vous effectuez au moment de l'évaluation, ainsi que l'inférence sur la base de test.

Procédure de soumission

- Ouvrez le notebook `solution.ipynb` et exécutez toutes les cellules. Vérifiez qu'il écrit `answers.json` dans le répertoire de travail (ou dossier dans lequel vous travaillez), avec une permutation pour chaque dialogue de `dataset/test_public/` (100 dialogues). Au moment de l'évaluation, le notebook est exécuté à nouveau sur l'ensemble de test (qui est caché), et le fichier `answers.json` qu'il y produit est évalué.
- Améliorez la solution si vous le souhaitez — ou ne le faites pas ; le baseline suffit à valider le pipeline.
- Ouvrez l'onglet Git dans la barre latérale gauche de JupyterLab.
- Ajoutez à la zone de préparation nommé **Stage** le fichier `solution.ipynb` (l'icône + à côté).
- Saisissez un message de commit et cliquez sur **Commit**.
- Cliquez sur l'icône de nuage avec une flèche vers le haut pour effectuer le push.
- Revenez sur cette page du *challenge* et cliquez sur **Submit**.

Soumettez exactement un fichier, nommé `solution.ipynb`.